

## 제 4 장 기하학적 처리와 변환

### 1. 기하학적 처리와 변환 과정이란?

기본적인 기하학적 변환에는 크기 조정, 이동, 회전 처리를 수반한다(그림 1). 기하학적 변환 처리란 픽셀의 위치를 새롭게 재 정렬하는 과정이지만 확대와 같은 영상 확대 과정은 없던 화소 값이 새롭게 생성되기도한다. 이러한 변환 과정을 입출력간의 매핑으로 설명되어 질 수 있다. 변환 과정에 의한 새로운 화소 생성은 보간법에 의해 얻어진다.

기하학적인 변환이 필요한 이유는 작은 영상을 확대하여 관찰해야는 경우라든지, 여러 영상을 다른 시각에서 촬영한 영상들을 정렬(registration)해야한다든지, 큰 영상을 보기 좋은 크기로 축소해야하는 경우라든지, 모자이크(mosaic) 영상 편집에서 영상을 이동시켜야 하는 경우에 필요하다.

기하학적인 변환의 기본 형태에는 **선형 기하 연산**(affine 변환)-평행이동, 회전, 크기(scaling) 변환- 으로 곡선이 전혀 없는 영상만을 대상으로하며 **비선형 기하 처리 연산**에는 워핑(warping)과 모핑(morphing)변환 - 처리될 영상을 찌그러뜨리고 구부리는것 과 같은 변환 과 모핑 변환 - 등이 있다.

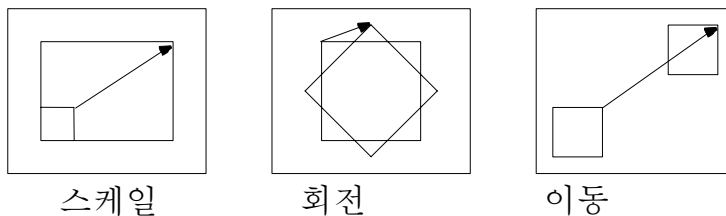


그림 1 기본적인 기하학적 처리:스케일,회전,이동

#### ◆ 순방향/역(inverse)방향 매핑

원 영상을 기하학적인 변환의 입력으로 작용하여 처리 영상이 얻어지는 과정을 순방향 매핑이라고하며 그 역 과정을 역방향 매핑이라고한다. 순방향 매핑시 2가지 문제점이 존재한다. 순방향 매핑에 의해 결과 영상의 모든 화소를 cover할 수 없는 경우 빈 곳(hole)이 발생하게 되며 경우에 따라서는 여러개의 입력 화소가 하나의 결과 영상 화소로 매핑이 이루어지는 중첩(overlap) 현상이 발생할 수 있다. 중첩이 되는 경우 어느 입력 화소를 선택할 것인가가 문제가 된다.

이같은 순방향 매핑의 문제점을 해결하기 위한 한 방법이 역 방향 매핑을 사용하는 것이다. 역방향 매핑 방법은 결과로 주어질 영상의 화소를 순차적으로 주어진 어떤 역 변환식에 의해 소스 영상의 어느 화소가 결과 영상의 화소를 생성하는데 사용할 것인가를 계산한다.

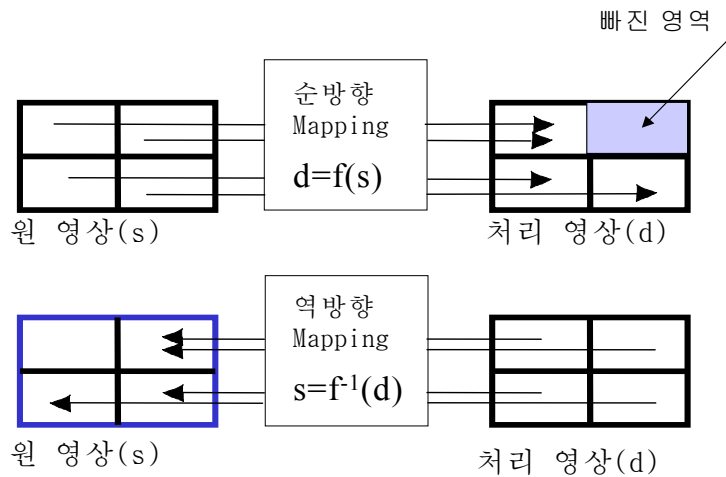


그림 2 전방향 과 역방향 사상(projection)

## 2. 보간(Interpolation) 기법

영상의 크기나 회전으로 인한 새로운 위치의 화소값을 결정하는 방법이 보간법에 의해 수행된다. 보간이란 정의된 화소간에 값을 결정하기 위하여 영상 데이터를 다시 표본화하는 과정이다. 매핑이 소숫점 화소(fractional pixel) 좌표를 나타내게 될 때 문제가 발생할 수 있다. 예를 들어 원시 좌표 위치를 정의하는 변환이 다음과 같은 식에 의해 주어지는 경우 주소를 읽거나 쓸 때 문제가 발생한다.

$$X_{\text{dest}} = 2 * X_{\text{source}}$$

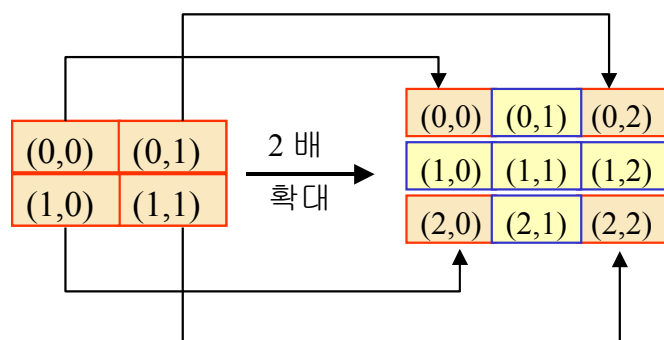
$$Y_{\text{dest}} = 2 * Y_{\text{source}}$$

$X_{\text{source}}[0][0]$ 은  $X_{\text{dest}}[0][0]$ 로 매핑이 되지만  $X_{\text{dest}}[1][1]$ 은  $X_{\text{source}}[0.5][0.5]$ 가 되어 어느 좌표와 매핑이 되지 않고 채워지지 않은 빈 공간으로 남는다. 이같은 픽셀의 좌표값  $[1][1]$ 은 원영상에 존재하지 않는다.

이같은 문제를 해결하기 위해 **보간 기법**이 필요하다. 보간 기법은 픽셀간에 놓일 주소에 대한 값을 생성하는 과정이라고 볼 수 있다. 고려해야할 사항은 보간 과정(보간 필터링)을 통해서 원화상보다는 화질이 떨어지며 처리 시간 비용이 추가된다. 자주 사용되는 보간 기법으로 최근접 이웃 화소 보간법, 평균 보간법(양선형 보간법), 고등차수 보간법, B-spline 보간법 등 다양하다.

### ◆ Nearest neighbor(최근접 이웃) 화소 보간 기법

출력 화소의 새롭게 생성된 주소에 가장 가까운 픽셀을 입력 영상에서 찾아 출력 픽셀로 할당하는 이 방법은 새로운 픽셀을 생성하지 않고 모든 출력 픽셀이 입력 픽셀의 집합으로부터 얻어지므로 결과의 픽셀 위치가 최대  $1/\sqrt{2}$  크기의 오류를 갖을 수 있다.



최대거리오차: 점(0,0) 에서 점(1,1)의 거리 오차

$$\sqrt{(0-1)^2 + (0-1)^2} = \sqrt{2} \text{ at pixel}(1,1)$$

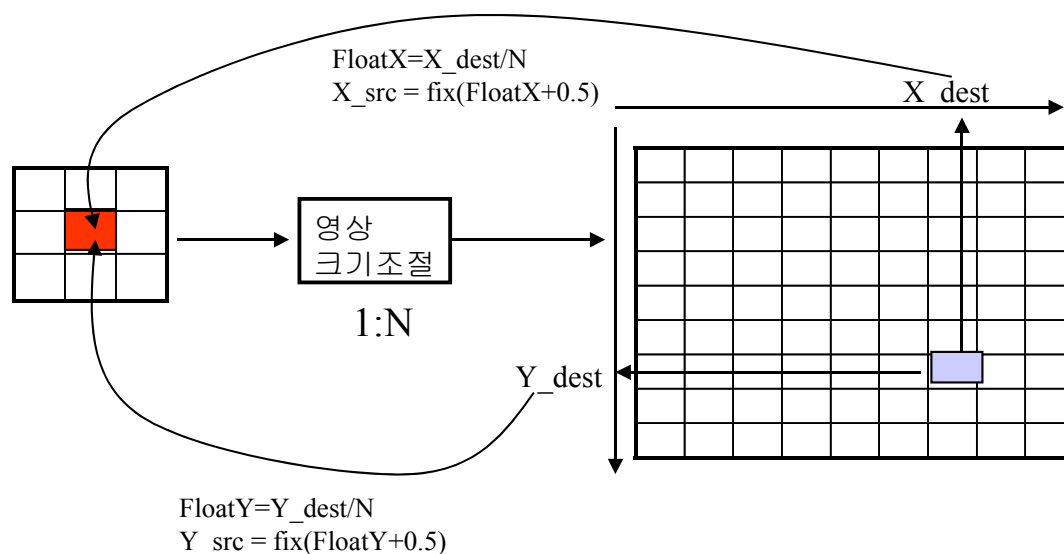
#### ❖ NN 보간 알고리즘 Pseudocode: (그림 참조)

```
floatX = NN_mapping(X_dest);
floatY = NN_mapping(Y_dest);
X_source = (fix) (floatX+0.5);
Y_source = (fix) (floatY+0.5);
```

여기서 NN\_mapping은 NN 보간 함수이다. 예를들어 NN\_mapping 함수를 1/scale\_factor 인 함수 라고하면 위 알고리즘의 수행 결과는 (scale\_factor=2 인경우)

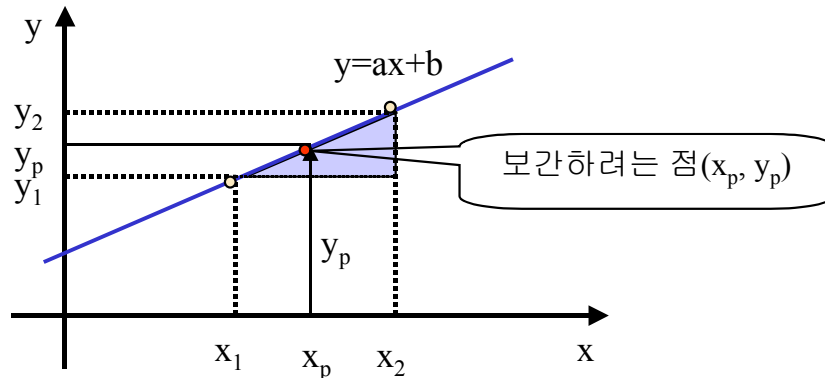
일반적으로 NN\_mapping = 1/SF=1/N.

| 출력 화소  |        | 입력 화소 |       |
|--------|--------|-------|-------|
| X_dest | Y_dest | X_src | Y_src |
| 0      | 1      | 0     | 1     |
| 1      | 0      | 1     | 0     |
| 1      | 1      | 1     | 1     |
| 1      | 2      | 1     | 1     |
| 2      | 1      | 1     | 1     |



### ◆ 양선형 (Bilinear) 보간 기법

양선형 보간 기법은 이미 알려진 두점의 값으로부터 그 사이에 놓인 임의의 위치에서의 값을 찾는 데 사용된다. 좌표값  $(x_1, y_1)$  과  $(x_2, y_2)$  이 주어 졌고 선상의 임의 위치에서 점  $(x_p, y_p)$  의 값을 구하고자 하면 삼각형의 닮음 꼴을 사용하여 푼다.



$$(x_2 - x_1) : (y_2 - y_1) = (x_p - x_1) : (y_p - y_1)$$

$$\frac{(x_2 - x_1)}{(y_2 - y_1)} = \frac{(x_p - x_1)}{(y_p - y_1)} \rightarrow y_p - y_1 = a(x_p - x_1)$$

$$slope = a = \frac{(y_2 - y_1)}{(x_2 - x_1)}$$

따라서 임의의 점  $x_p$  에서  $y_p$  를 구하는 것은 위 식으로부터 다음과 같다.

$$y_p = a(x_p - x_1) + y_1$$

그러나 영상을 확대한 경우 빈 홀을 채우기 위해 양선형 보간 기법의 가장 단순한 형태는 가장 이웃한 2x2 화소의 가중(weighted) 평균 값을 취한다. 양선형 보간은 3번 선형 보간이 필요하다.

$$y = b + ax, \quad b = \text{offset} = \text{NW}, \quad a = \text{기울기} = (\text{NE} - \text{NW})/1$$

삼각형의 닮음꼴을 사용하여 푼다(그림 4.3 참조).

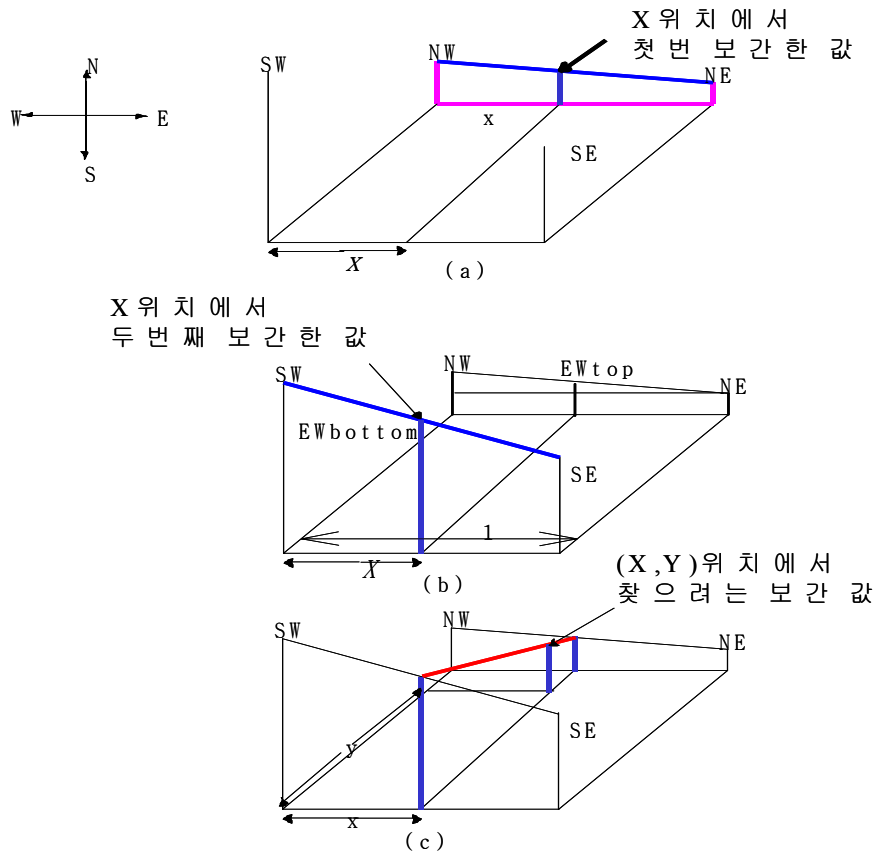


그림 9 양선형 보간은 어떻게 수행되는가 (a) 첫 번째 보간 (b) 두 번째 보간 (c) 최종 보간

#### ❖ 양선형 보간 알고리즘 pseudocode:

// 원하는 (X1,X2), Y 위치의 좌표 지정 혹은 설정

$x_{new} = X1$ ; //북동서 지점

$x_{sew} = X2$ ; //남동서 지점

// 첫 번째 보간: 북동서축으로의 보간

$(NW-NE):(EW_{top}-NW) = (-1):x_{new}$

$\rightarrow EW_{top} = NW + x_{new} * (NE - NW)$ ;

$x_{new} = \text{floatx} - \text{floor}(x)$ ;

// 두 번째 보간: 남동서축으로의 보간

$(SW-SE):(EW_{bottom}-SW) = (-1):x_{sew}$

$\rightarrow EW_{bottom} = SW + x_{sew} * (SE - SW)$ ;

$x_{sew} = \text{floatx} - \text{floor}(x) = x_{new} = x$ ;

// 세 번째 보간: 남북축으로의 보간

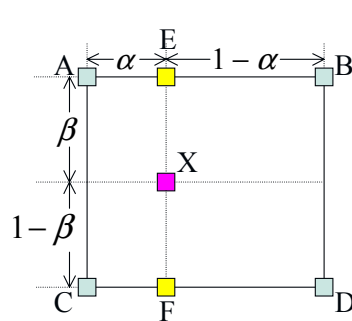
$(EW_{bottom}-EW_{top}):(dest\_pixval-EW_{top})=(-1):(y)$

$\rightarrow dest\_pixval = EW_{bottom} + y * (EW_{top} - EW_{bottom})$ ;

$Y = \text{floaty} - \text{floor}(y)$ ;

// 리턴되는 값: 위치  $x_{new}$ ,  $x_{sew}$ ,  $y$  정보 와 이 위치에서의 크기 정보( $dest\_pixval$ )

## \*도해적 설명



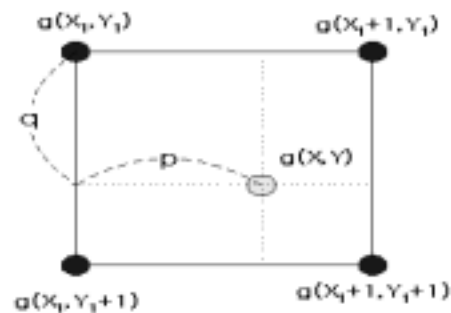
- 원시 화소
- X축 보간 화소
- 최종 보간 화소

$$E = (1-\alpha)A + \alpha B = A + \alpha(B-A)$$

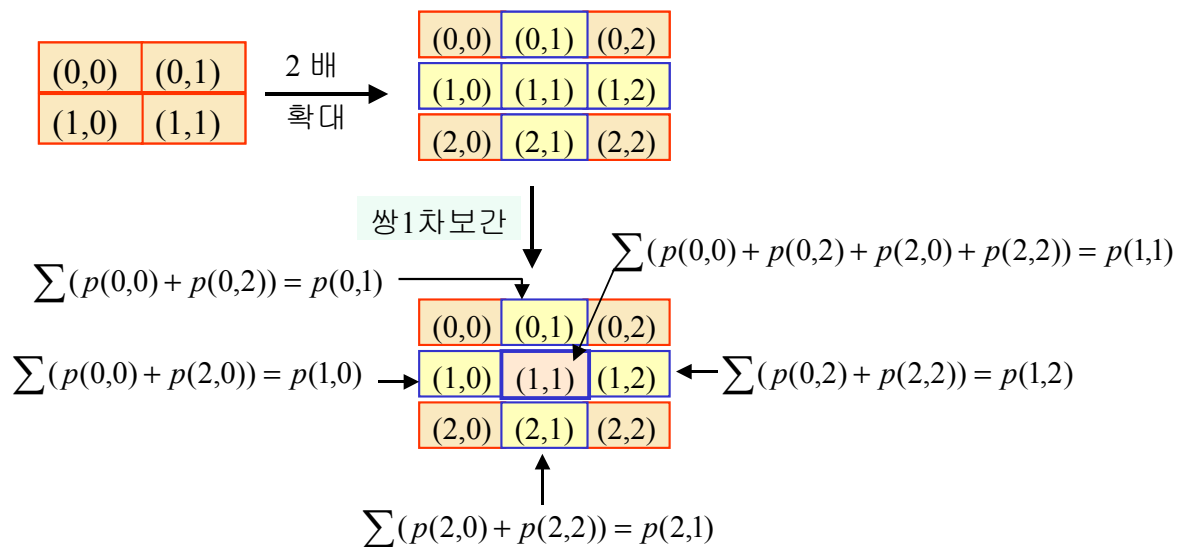
$$F = (1-\alpha)C + \alpha D = C + \alpha(D-C)$$

$$X = (1-\beta)E + \beta F = E + \beta(F-E)$$

$$\begin{aligned} g(X, Y) &= pqg(X_1, Y_1) + p(1-q)g(X_1, Y_1+1) \\ &\quad + (1-p)qg(X_1+1, Y_1) \\ &\quad + (1-p)(1-q)g(X_1+1, Y_1+1) \end{aligned}$$



양선형 보간 방법을 도해적으로 나타내보이면 아래 그림과 같다. 먼저 2x2 화소들을 2배로 확대하고 난 후 빈 홀에 해당하는 화소들을 찾기 위하여 주변의 화소의 평균 값을 찾는다.



## ◆ 고등차수 보간 기법

NN 보간법은 한 개의 입력 화소를 필요로 하고, 양선형 보간 기법은 4개의 입력 화소를 필요로 한다. 고등차수 보간 기법은 주변의 더 많은 입력 화소들로부터 보간된다.

Cubic 컨볼루션 유형의 보간 기법 과 B-spline 유형의 보간 기법은 보간에 필요한 모형 함수를 요한다. 간단한 1차원 보간 함수의 예를 그림에 보였다. 보다 일반적인 고차수 보간 함수는 그림 (a)에 보였다.

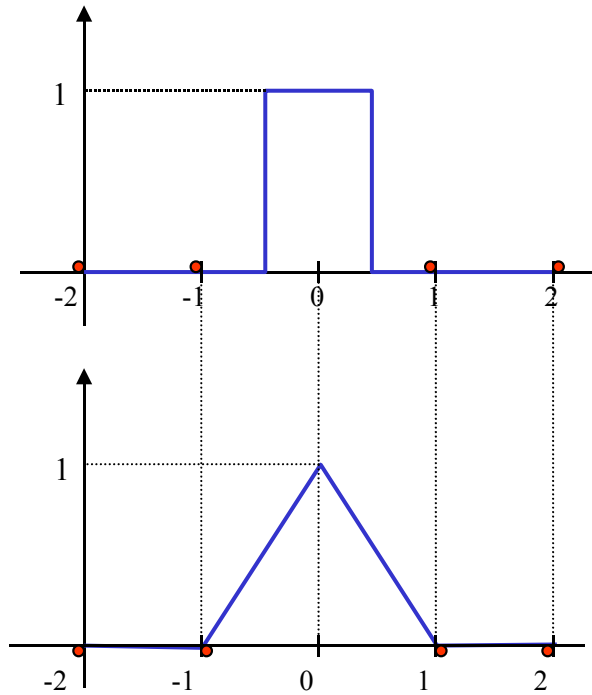


그림 14 1차원 보간 함수들

(a) 최근접 이웃 화소 보간법

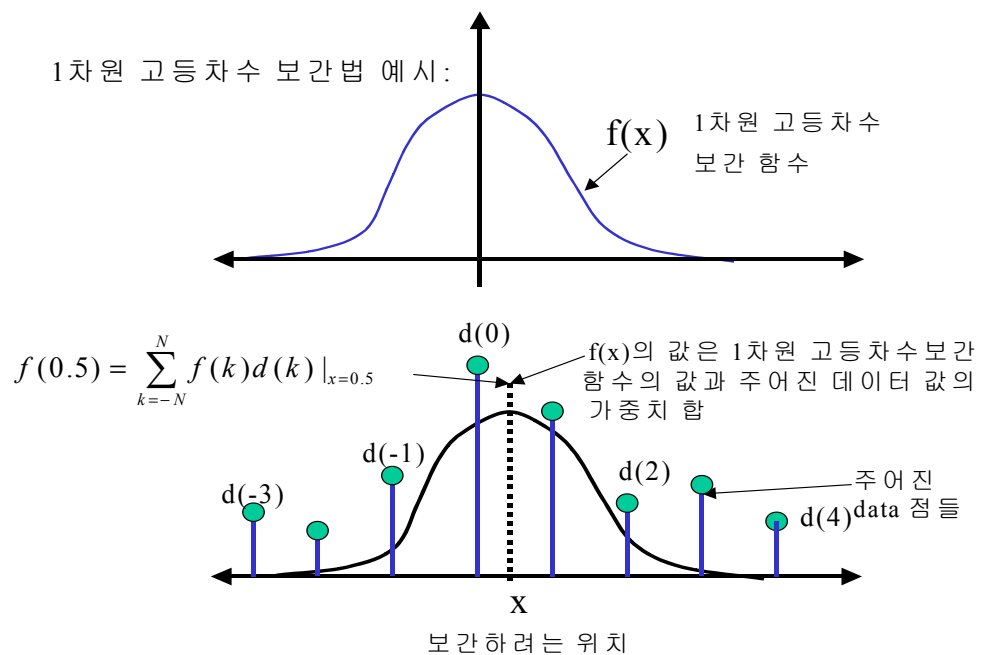
(b) 선형 보간법

1차원 보간을 각 행에 대하여 4번 수행하고 각 열에 대하여 4번 수행하여 2차원 보간으로 쉽게 확장될 수 있다. 각 행의 연산 결과는 하나의 미지(혹은 알고자는 좌표의  $x$  위치)의  $x$  좌표에서의 하나의 가중치 값을 생성한다. 나머지 행에 대하여 1차원 보간을 행하면 4개의 생성된 가중치 값을 얻게 된다. 그리고 나서 열 방향으로 하나의 최종 화소를 보간하기 위해 행방향의 4개의 값을 사용한다. 이렇게 나머지 열에 대하여 수행하면 4개의 생성된 가중치 값을 얻게 된다. Bi-cubic 보간 기법에 의한 출력 화소의 결정은 가장 이웃한  $4 \times 4$  화소들의 가중 평균 값을 취한다.

## \* Bi-cubic 보간법의 도해적 설명

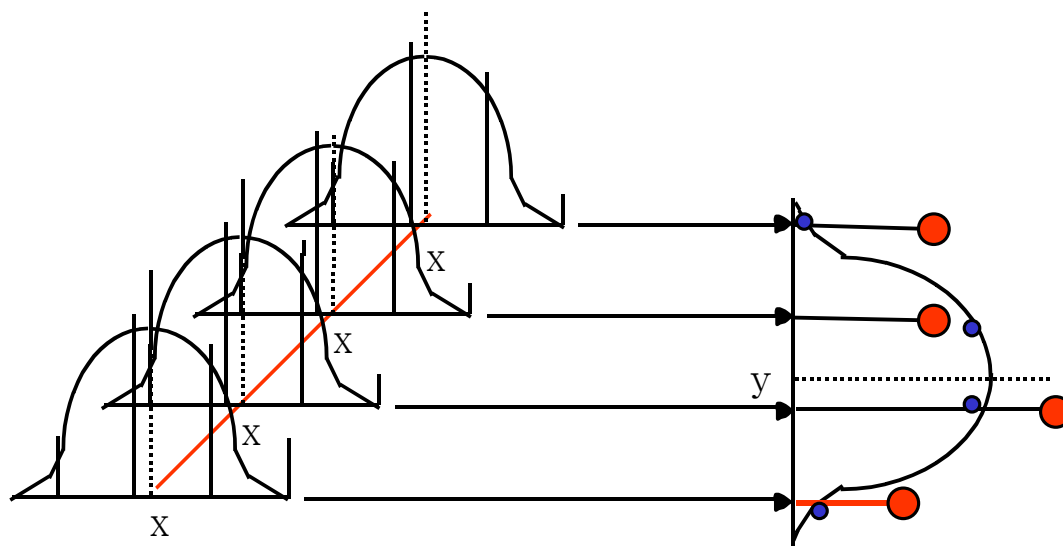
### 1차원 보간법:

1차원 고등차수 보간법 예시:



### 2차원 보간법:

모형함수의 예: (2차원)





# ◆ cubic convolution 보간 기법

1차원 cubic convolution 함수의 예:

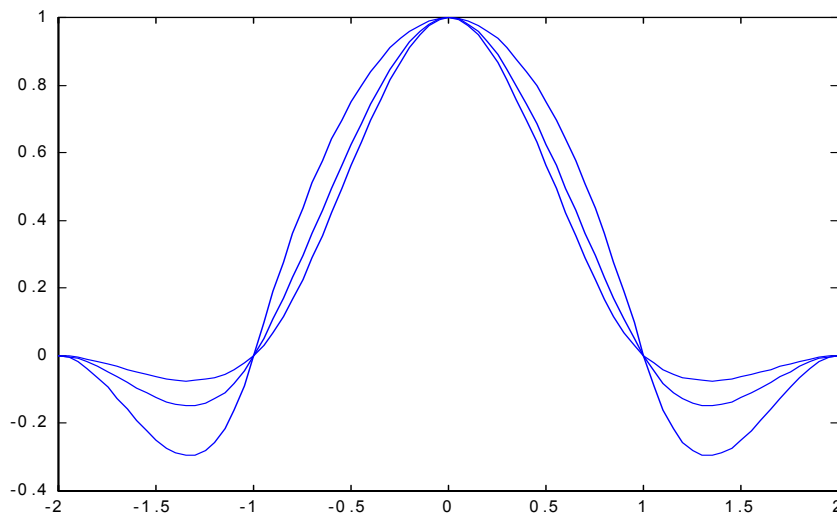
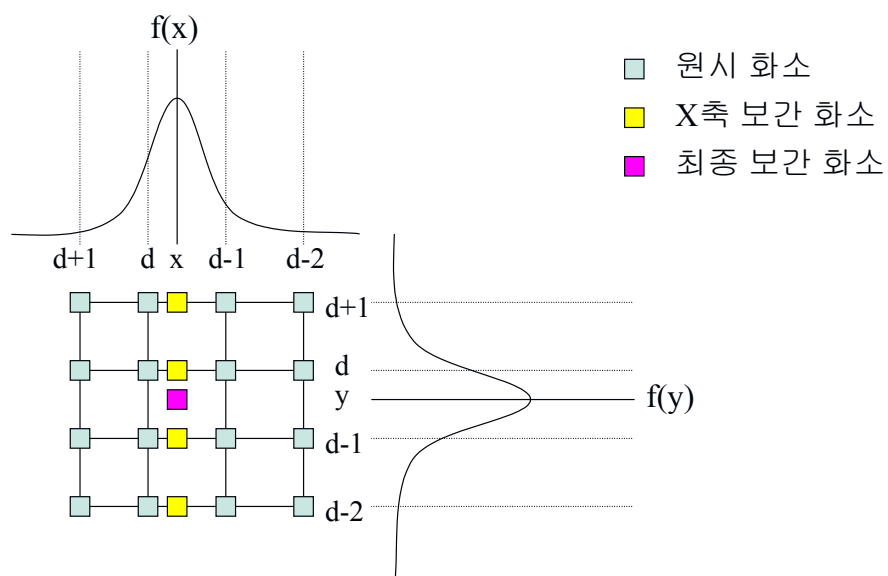
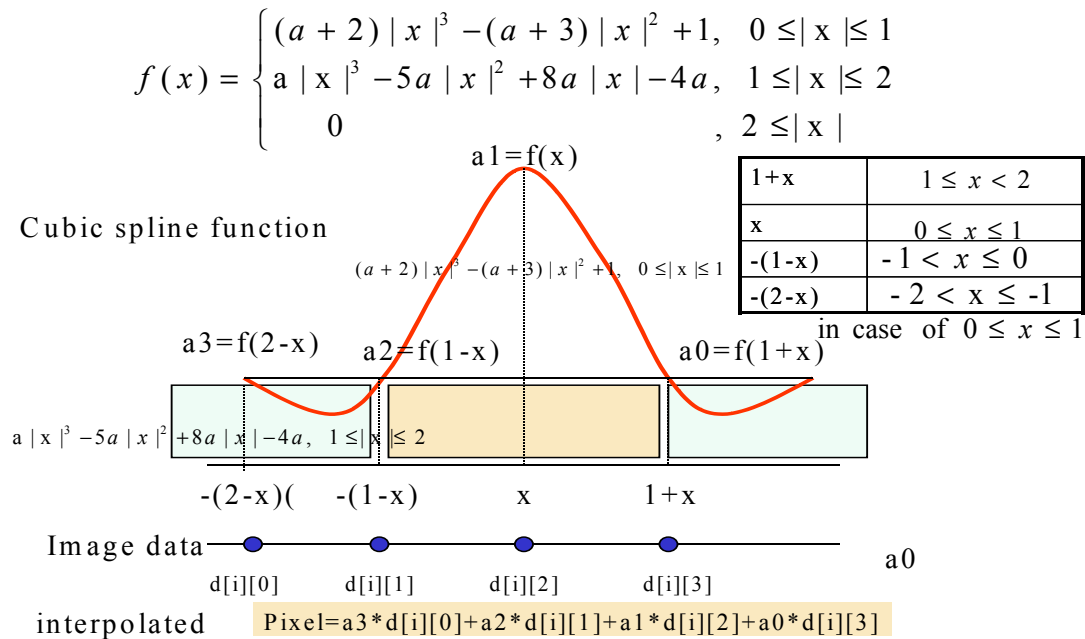


그림 A  $a = -0.5, -1.0, -2.0$ 에 대한 3차 보간 함수의 모양

$$f(x) = \begin{cases} (a+2)|x|^3 - (a+3)|x|^2 + 1, & 0 \leq |x| < 1 \\ a|x|^3 - 5a|x|^2 + 8a|x| - 4a, & 1 \leq |x| < 2 \\ 0, & 2 \leq |x| \end{cases}$$

## \* Cubic spline 함수에 의한 보간 방법의 도해적 설명





프로그램 수행의 예 (교과서 프로그램 list4.3): 3차 회선 보간법

알고리즘 : 16개의 좌표점에서 화소값이 필요하다.

| col \ row | 0     | 1     | 2     | 3     | 4 | 5 | 6 | 7 |
|-----------|-------|-------|-------|-------|---|---|---|---|
| 0         | (0,0) | (0,1) | (0,2) | (0,3) |   |   |   |   |
| 1         |       | (1,1) |       |       |   |   |   |   |
| 2         |       |       |       |       |   |   |   |   |
| 3         | (3,0) | (3,1) | (3,2) | (3,3) |   |   |   |   |
| 4         |       |       |       |       |   |   |   |   |
| 5         |       |       |       |       |   |   |   |   |
| 6         |       |       |       |       |   |   |   |   |
| 7         |       |       |       |       |   |   |   |   |

예를 들면 4x4 image block이 8x8 image block으로 가로, 세로 2배씩 확대되어 보간된다. (1,1) pixel이 보간해야할 위치가 된다.

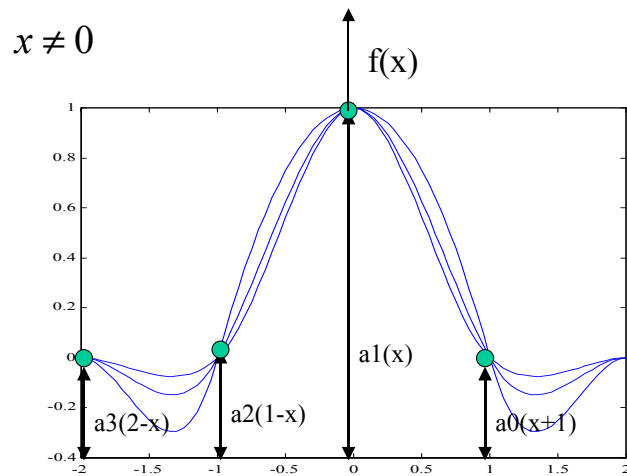
### ❖ 3차 컨볼루션 보간법의 Pseudocode:

```

read the reduced image
for every row of the reduced image
  for every col of the reduced image
    get a 4x4 sub image block of the the reduced image
    for a given sub image block
      perform cubic interpolation for newly chosen position (x,y)
    endfor
  endfor
endfor
    
```

endfor

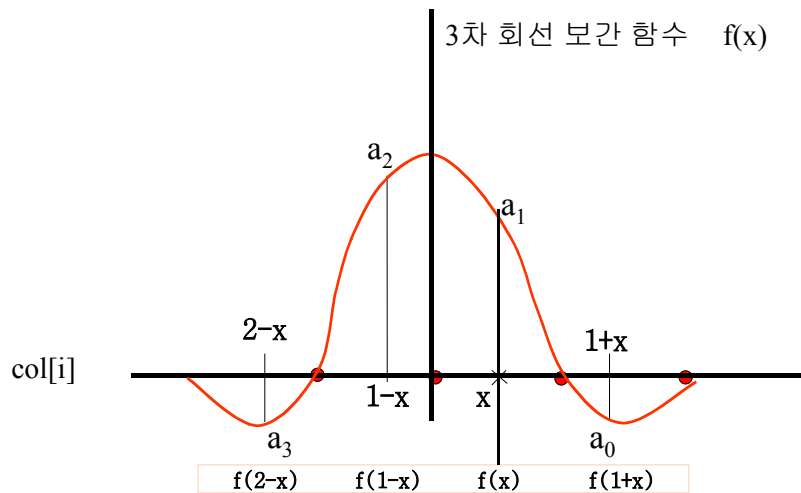
save the interpolated blowup image



#### ✚ C code:

```
/* cubic interpolation start.... */
xoffset=yoffset=0; /* blowup this much */
for (i=0; i<rows; i++)
{
    for (j=0; j < cols; j++)
    {
        for (ii=0; ii<4; ii++)
            for (jj=0; jj<4; jj++)
            {
                /* get 4x4 image block */
                image[ii][jj]=in->image[i+ii][j+jj];
            }

        jx=iy=0;
        for (y=0.0; y < 1.0; y += 1.0/8.0) //2배 확대
        {
            for (x=0.0; x < 1.0; x += 1.0/8.0) //2배 확대
            {
                out->image[iy+yoffset][jx+xoffset] =
                cubic_interpolation(image, x, y, a);
                jx++;
            }
            iy++;
            jx=0;
            xoffset +=2; /* blowup the original image in the x-direction using
cubic interp */
        }
        yoffset +=2; /* blowup the original image in the y-direction using cubic
interp */
        xoffset = 0;
    }
}
/* cubic start end.....*/
```



보간할 위치  $x=0.5$

$$\text{col}[i] = \text{image}[i][0] * a_0 + \text{image}[i][1] * a_1 + \text{image}[i][2] * a_2 + \text{image}[i][3] * a_3$$

//x 방향에대한 보간:

$a_0 = f(1+x)$ ,  $a_1=f(x)$ ,  $a_2=f(1-x)$ ,  $a_3=f(2-x)$ 라고 두면

$a_1=((a_2)*x-(a_3))*x*x+1$  등등을 계산하고 서브영상 블록에대하여

$\text{col}[i]=a_0*\text{image}[i][0]+a_1*\text{image}[i][1]+a_2*\text{image}[i][2]+a_3*\text{image}[i][3]; i=0, 1, 2, 3$

//y 방향에대한 보간:

$a_0, a_1, a_2, a_3$ 를 x방향의 보간계산과 같은 방법으로 구한다

$\text{pixel} = \text{col}[0]*a_0+\text{col}[1]*a_1+\text{col}[2]*a_2+\text{col}[3]*a_3;$

## 실습: 3차원 컨볼루션 보간 실습

## C Program Source

```

/*****
*Main routine: Plist4_3.c
subroutines req'd:
    cubic_interpolation.c (chapter 4 list4_3.c)
    iplib.c *
* Desc: This program performs Cubic Interpolation.
*       This routine performs cubic interpolation of the downsized image
*       and make a upsized image by a factor of 4.
*       Need to modify the parameters xoffset and yoffset value
* Modified by D.C. Park
* Date : Feb. 1, 1998      ver. 0.0
*
*****/

#include <stdio.h>
#include <string.h>
#include <malloc.h>
#include <process.h>
#include "ip.h"

extern void write_pnm(image_ptr ptr, char filein[], int rows,
                     int cols, int magic_number);
extern image_ptr read_pnm(char *filename, int *rows, int *cols,
                          int *type);
extern unsigned char cubic_interpolation(unsigned char image[4][4],
                                         float x, float y, double a);

void main(int argc, char *argv[])
{
    char filein[100];          /* name of input file */
    char fileout[100];         /* name of output file */

    FILE *fp;
    int rows, cols;           /* image rows and columns */
    image_ptr buffer;         /* pointer to image buffer */

    int type;                 /* what type of image data */
    int i, j, ii, jj;
    float x, y;
    double a=-1.0;
    unsigned char image[4][4];
    typedef struct
    { unsigned char image[128][128];
    } INs;
    INs *in;
    typedef struct
    { unsigned char image[512][512];
    } OUTs;
    OUTs *out;
    int iy, jx, xoffset, yoffset;

    /* set input filename and output file name */
    if(argc == 3)
    {
        strcpy(filein, argv[1]);
        strcpy(fileout, argv[2]);
    }
    else {
        printf("Input name of input file[ex: bowl.pgm]?");
        gets(filein);
        printf("\nInput name of output file[ex: bowl.pgm]?");
        gets(fileout);
    }
}

```

```

    /* read the reduced original image into the buffer*/
    buffer = read_pnm(filein, &rows, &cols, &type);
    in=(INs *)malloc(sizeof(INs));
    out=(OUTs *)malloc(sizeof(OUTs));

    printf("filein name is %s with rows %d, cols %d, type %d\n", filein, rows, cols,
type);

    if(type != PGM) {
        printf("Cubic Interpolation is performed only PGM files!!\n");
        exit(1);
    }

    /* 1-dim array to 2-d array conversion */
    for(i=0; i<rows; i++)
        for(j=0; j< cols; j++)
        {
            in->image[i][j]=buffer[i*cols+j];
        }

    /* open output file for writing*/
    if((fp=fopen(fileout, "wb")) == NULL)
    {
        printf("Unable to open %s for output for writing\n", fileout);
        exit(1);
    }

    /* write out header */
    fprintf(fp, "P%d\n%d %d\n 255\n", type, cols*2, rows*2);
    /* cubic interpolation start.... */
    xoffset=yoffset=0; /* blowup this much */
    for (i=0; i<rows; i++)
    {
        for (j=0; j < cols; j++)
        {
            for (ii=0; ii<4; ii++)
                for (jj=0; jj<4; jj++)
                {
                    image[ii][jj]=in->image[i+ii][j+jj]; /* get 4x4 image block */
                }
            jx=iy=0;
            for (y=0.0; y <1.0; y += 1.0/8.0) //8개 데이터 보간
            {
                for (x=0.0; x < 1.0; x += 1.0/8.0) //8개 데이터 보간
                {
                    out->image[iy+yoffset][jx+xoffset] =
                        cubic_interpolation(image, x, y, a);

                    jx++;
                }
                iy++;
                jx=0;
            }
            xoffset +=4; /* blowup the original image in the x-direction
                        using cubic interp */
        }
        yoffset +=2; /* blowup the original image in the y-direction using
        cubic interp */
        xoffset = 0;
    }

    /* cubic start end..... */
    for(i=0; i<rows*2; i++)
        fwrite(out->image[i], sizeof(char), cols*4, fp);

    IP_FREE(buffer); /* memory free */
    free(in);
    free(out);
}

```

◆ 프로그램 수행 결과:



그림 24 256x256크기의 원영상



그림 25 128x128 크기의 원영상



그림 26 cubic interpolation(list4.3)를 사용하여 얻은 결과(256x256크기로 확대)

■ B-spline 보간 기법

cubic spline 방법과 유사함(생략)

### 3. Image scaling(영상의 확대 및 축소)

**정의식:** 좌표점 (x,y) 위치의 화소가 원점을 중심으로 가로방향으로 a 만큼, 세로 방향으로 b 만큼 확대될 경우

$$\begin{bmatrix} X \\ Y \end{bmatrix} = \begin{bmatrix} ax \\ by \end{bmatrix} \quad \begin{array}{l} \text{If } a \text{ or } b > 0, \text{ then 확대} \\ \text{else 축소} \end{array}$$

영상을 원하는 임의의 크기로 확대하거나 축소할 필요가 있다. 기억해야할 중요한 사항은

- 1) 원 영상의 해상도를 image scaling 작업에 의해 결코 향상 시킬 수 없다.
- 2) 영상을 스켈링할 때 항상 원 영상을 참조하지 않으면 스케일된 결과의 영상의 화질은 항상 저하된다.

영상 확대 : 픽셀값은 각방향 2배의 크기로 복사되어 확대한다.

영상 축소 : 픽셀값은 각방향 1/2배의 크기로 축소된다.

프로그램 실행 결과(list4.5):



그림 28 original  
128x128 image



그림 29 x방  
향 으로 1/2  
축소



그림 30 scale up by 2 in x-direction



## 영상 축소 기법

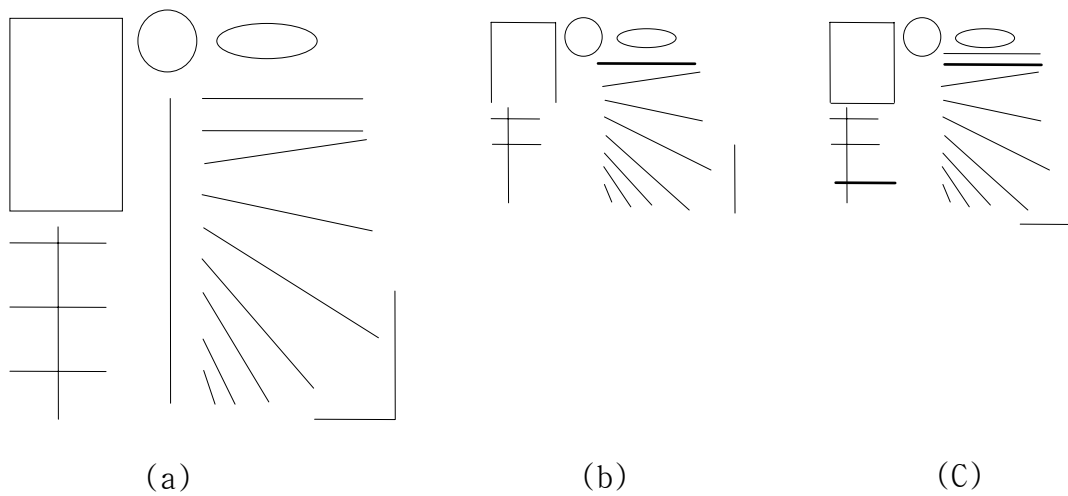


그림 4.12 (a)원래 영상 (b) 2에 의한 서브샘플링에 의해서 축소된 영상  
(c)흐리게 만들고 나서 2만큼 축소된 영상

그림 31

## ■ 중앙값에 의한 축소

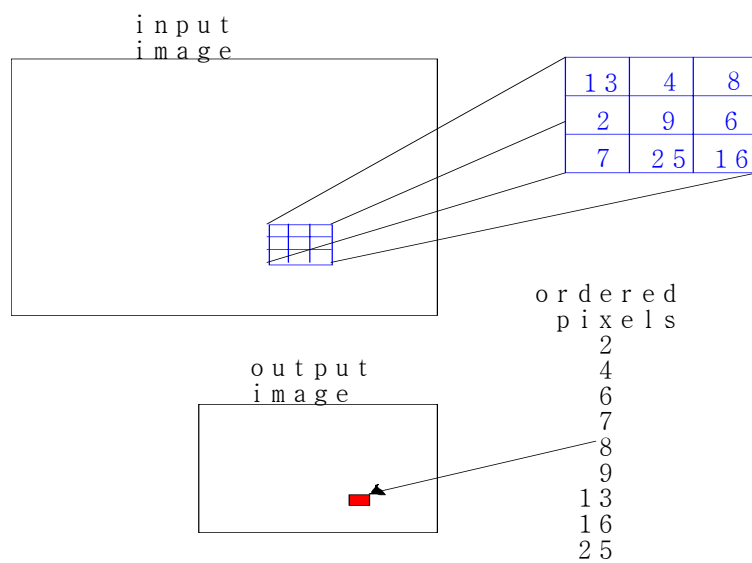


그림 4.32 중간수 표현에 의한 축소

## ■ 평균에 의한 축소

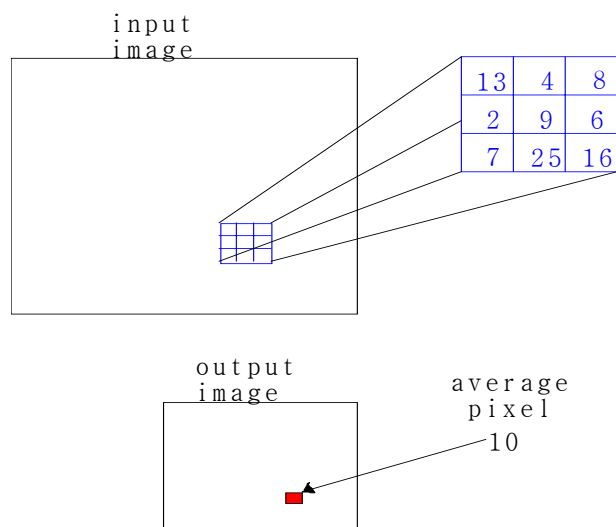
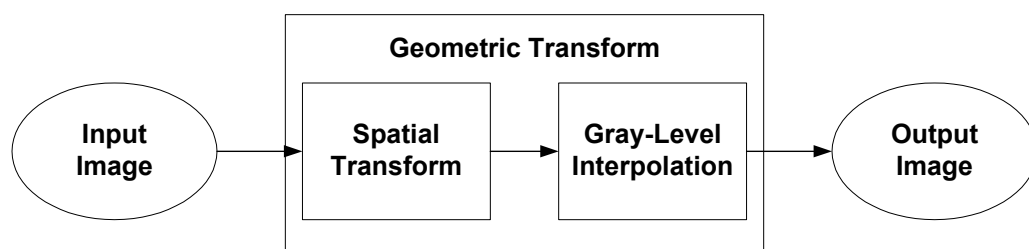


그림 4.33 평균 표현에 의한 축소

## 4. 회전 변환



회전을 위한 알고리즘(Euler 공식)

$(X_1, Y_1)$ 은 원 영상의 좌표점이고  $(X_2, Y_2)$ 는 변환후의 좌표점이다.

### 1) 원점에 대한 회전

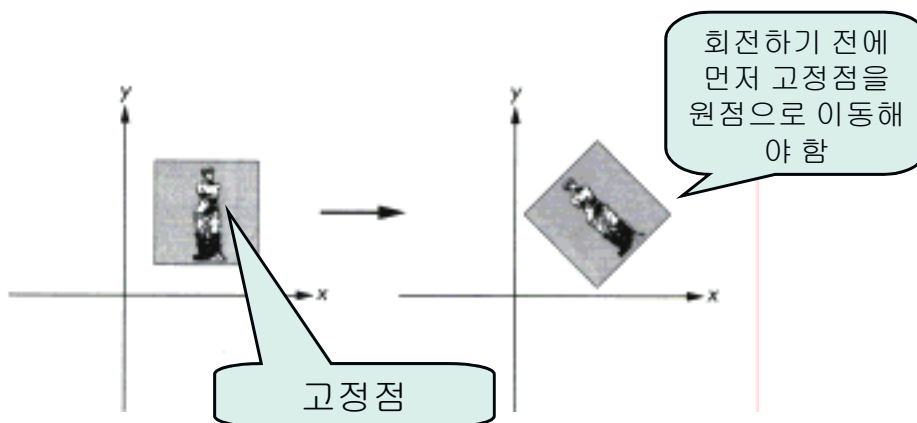
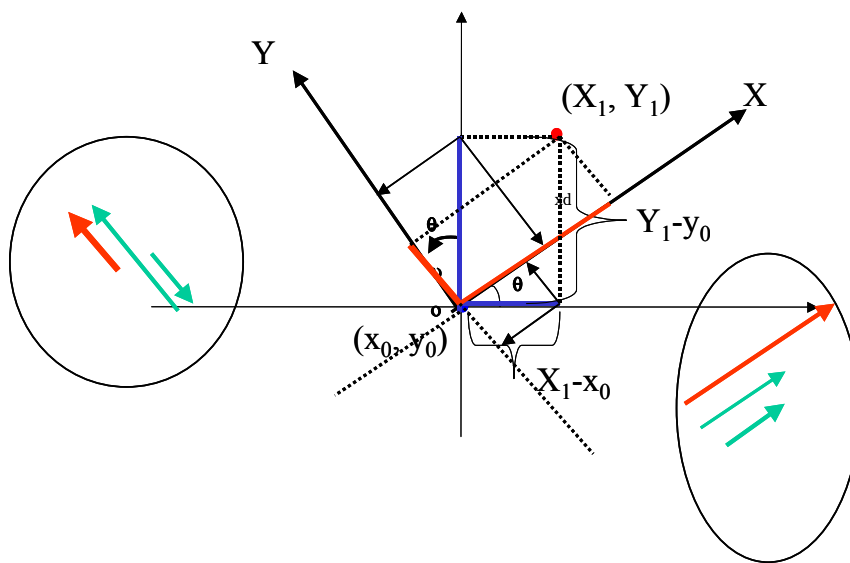
원점  $(0, 0)$ 에 대한 각  $\theta$ 만큼 반시계방향으로 회전했을 때 좌표점간의 관계는 다음 식으로 주어진다.

$$\begin{bmatrix} X_2 \\ Y_2 \end{bmatrix} = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} X_1 \\ Y_1 \end{bmatrix}$$

### 2) 중앙 픽셀에 대한 회전

이미지의 중앙 화소값에 대한 다음 식에 의한 회전은 중앙 화소의 좌표점  $(x_0, y_0)$ 에 대한 반시계 방향으로 회전에 대응한다.

$$\begin{bmatrix} X_2 - x_0 \\ Y_2 - y_0 \end{bmatrix} = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} X_1 - x_0 \\ Y_1 - y_0 \end{bmatrix}$$



#### ✦ C++ source code: 이미지 회전

```
void CTestDoc::Rotate()
{
    double i=0, j=0, m=0, n=0, X=0, Y=0;
    int y=0, x=0, left=0, right=0;
    double seta=0, pi=3.1415926535;

    seta= pi/4;
    for(y=0; y<256; y++){
```

```

        for(x=0; x<256; x++){
            i=((y-128)*sin(seta))+128;
            j=(x-128)*cos(seta);
            X=(j+i);//vector sum
            if(X>255)X=255;
            if(X<0)X=0;
            m= -((x-128)*sin(seta))+128;
            n=(y-128)*cos(seta);
            Y=(n+m);//vector sum
            if(Y>255)Y=255;
            if(Y<0)Y=0;

            m_ResultImg[(int)Y][(int)X]=m_OpenImg[y][x];
        }
    }

    //////////////////////////////////평균 값을 이용한 보간법////////////////////////////////////
    for(y=1; y<255; y++){
        for(x=1; x<255; x++){
            left=m_ResultImg[y][x-1];
            right=m_ResultImg[y][x+1];
            if(left!=0 && right!=0 &&m_ResultImg[y][x]==0)
            {
                m_ResultImg[y][x]=(left+right)/2;
            }
        }
    }
    //////////////////////////////////평균 값을 이용한 보간법////////////////////////////////////
}

```

---

## ■ 이동 (translation)

영상의 일부분을 같은 영상내의 다른 곳으로 이동시키는 것을 말한다. 영상의 일부분 영역의 이동은 double buffering 기법을 사용해야한다.

\*더블 버퍼링 :

When a graphic is complex or is used repeatedly, you can reduce the time it takes to display it by first rendering it to an off-screen buffer and then copying the buffer to the screen. This technique, called double buffering, is often used for animations.

## ■ 미러링(mirroring)

영상의 x축 혹은 y축에 대한 단순 reflection (뒤집기)이므로 화소들의 단순한 이동에 해당되므로 보간 기능을 수행되지 않는다.

## 5. 프로그래밍 실습

[1] 양선형 보간에 의한 영상 확대

[2] 회전 변환- affine 변환[다이어로그박스 사용]

## ◎ Further Study

일반적으로 down sampling을 할 때는 anti-aliasing filter를 거친후 downsampling 과정이 이루어

진다. 반대로 up sampling을 할 때는 interpolation filter를 거쳐야 된다.

1) ITU-R (CCIR) 601 format의 표본화 포맷에서 휘도(Y) 와 색차(Cr, Cb)의 샘플링 위치를 설명하라

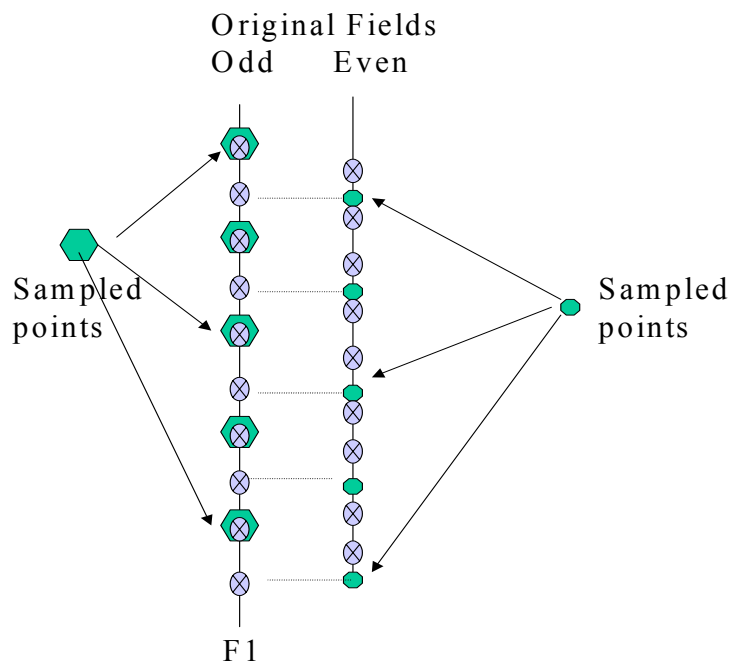
2) ITU-R (CCIR) 601(4:2:2)에서 4:2:0 format으로의 포맷 변환을 수행하라.

4:2:2 format : Y(720x480), Cr(360x480), Cb(360x480)

4:2:0 format : Y(720x480), Cr(360x240), Cb(360x240)

사용할 필터 : The following 7-tap vertical filter is used to pre-filter the FIELD1:  $[-29, 0, 88, 138, 88, 0, -29]/256$ . Then vertical subsampling by 2 is performed. The 4-tap vertical filter is used to decimate the FIELD2:  $[1, 7, 7, 1]/16$ . Then vertical subsampling by 2 is performed.

3) 위의 1) 과 2)를 참조하고 아래 샘플링 패턴을 참조하여 CCIR601 4:2:2 포맷에서 CCIR601 4:2:0 포맷으로 변환(다운 샘플링)하는 프로그래밍을 수행하는 코드를 작성하라.

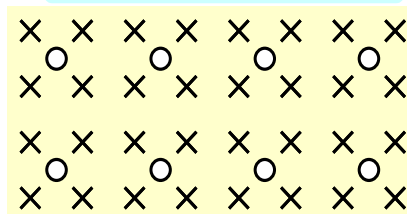
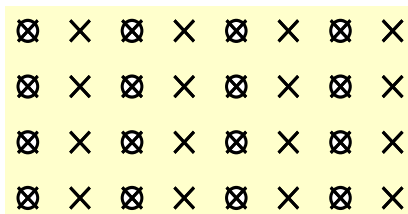


표본화 포맷:422(CCIR-601 권고)

표본화 포맷 : 420(Mpeg-1)

**4:2:2(CCIR 601 권고 표준)**

**4:2:0(mpeg-1 내부처리)**



표본화 포맷 : 420(Mpeg-2)

## 4:2:0(mpeg-2 내부처리)

